

الدليل التقني لمشروع Platinum

مرجع عربي لمناقشة مشروع التخرج — الإصدار الحالي يركّز على لوحة الإدارة وخدمة العملاء، إضافة إلى البنية المشتركة بينهما.

1. ملخص سريع يمكن قوله أمام اللجنة

Platinum هو تطبيق ويب لإدارة شركة عقارية/مقاولات. الواجهة الأمامية مبنية بـ React و Vite، وتتصل مع REST API مبنية على Laravel. قسّمنا التطبيق حسب أدوار المستخدمين، ووفّرنا لوحة خاصة للإدارة وأخرى لخدمة العملاء. استخدمنا Redux Toolkit لإدارة الحالة والطلبات غير المتزامنة، و Axios للاتصال بالـ API، و React Router لحماية المسارات والتنقل، و Laravel Echo مع Pusher للمحادثة الفورية. كما يدعم التطبيق العربية والإنجليزية، RTL وLTR، والوضعين الفاتح والداكن.

2. التقنيات واللغات المستخدمة

التقنية	استخدامها في المشروع
JavaScript ES6 +	منطق التطبيق، التعامل مع البيانات وAPI
JSX	بناء واجهات React
HTML5 وCSS3	هيكلية وتصميم الواجهات، Dark/Light، Responsive، RTL
JSON	صيغة البيانات المتبادلة مع REST API
React 19	بناء الواجهات على شكل Components
Vite 8	تشغيل بيئة التطوير وتجميع نسخة الإنتاج
React Router DOM 7	المسارات، Nested Routes، وحماية الصفحات
Redux Toolkit	Global State و Async Thunks وحالات loading/error
React Redux	ربط React بال Redux Store
Axios	إرسال HTTP requests وإعداد base URL والheaders
Laravel Echo + Pusher JS	استقبال رسائل الدردشة لحظيًا عبر WebSocket
Firebase	دعم الإشعارات و FCM device tokens
react-hot-toast	رسائل النجاح والخطأ السريعة
Lucide React	أيقونات موحدة وقابلة للتخصيص
Recharts	الرسوم البيانية في الصفحات التي تعرض إحصائيات
Leaflet + React Leaflet	الخرائط والمواقع في الوحدات التي تحتاج موقعًا جغرافيًا
Lodash	Utilities لمعالجة البيانات عند الحاجة
ESLint	اكتشاف أخطاء وجودة JavaScript و React
Babel / React Compiler	تحويل وتحسين كود React أثناء البناء

لماذا React؟

لأن المشروع يحتوي على لوحات متعددة وعناصر تتغير باستمرار مثل الجداول، النوافذ، الإحصائيات والدردشة. React يسمح بتقسيمها إلى Components قابلة لإعادة الاستخدام وتحديث الجزء المتغير فقط بدل إعادة بناء الصفحة كاملة.

لماذا Vite؟

لأنه سريع في التطوير، يدعم Hot Module Replacement، ويولد نسخة Production محسنة. متغيرات البيئة الخاصة بالواجهة تبدأ عادة بـ `VITE_` حتى يستطيع Vite قراءتها.

3. معمارية المشروع

اعتمدنا أسلوبًا قريبًا من **Feature-based Architecture**:

```
src/  
├─ app/           إعداد التطبيق، layouts، المسارات، الـ  
├─ shared/        i18n، theme، مشترك، مكونات عامة API  
└─ Rools/  
    ├─ admin/      صفحات وميزات الإدارة  
    └─ customerService/ صفحات وميزات خدمة العملاء
```

كل Feature تحتوي غالبًا على:

```
feature/  
├─ api/           payload وتحضير endpoints تعريف  
├─ model/         Redux slice و async thunks  
├─ styles/        الخاص بالميزة CSS  
└─ components أو pages
```

مسار البيانات في أغلب الميزات

```
ضغط المستخدم  
↓  
Component / Page  
↓ dispatch  
Async Thunk  
↓  
Feature API function  
↓  
Axios instance → Laravel REST API  
↓  
Response موحد  
↓  
Redux Slice (loading / success / error)  
↓  
إعادة رسم الواجهة تلقائيًا
```

ميزة هذا التقسيم أن الصفحة لا تعرف تفاصيل Axios، وطبقة الAPI لا تعرف طريقة العرض، و Slice مسؤول عن الحالة فقط. هذا يسهل الاختبار والصيانة.

4. طبقة الاتصال بالـ API

يوجد wrapper مشترك فوق Axios في `src/shared/api/crud.js` يدعم:

- `GET` , `POST` , `PUT` , `PATCH` , `DELETE` .
- JSON و `FormData` .
- إضافة `Authorization: Bearer <token>` تلقائيًا.
- إضافة `Accept-Language` حسب لغة الواجهة.
- توحيد النتيجة إلى `{ ok, status, message, data }` .
- تحويل أخطاء Laravel المختلفة إلى رسالة قابلة للعرض.
- عدم طباعة token الحقيقي في سجل الطلب.

JSON أم FormData؟

- نستخدم JSON عندما تكون البيانات object عادية أو تحتوي arrays منظمة، مثل شروط القرعة.
- نستخدم `FormData` عندما يتوقع الـ backend حقول form-data أو عندما تكون الميزة قابلة لدعم ملفات.
- لا نضع `Content-Type: multipart/form-data` يدويًا؛ المتصفح يضيف الـ boundary الصحيح.

المصادقة وحماية المسارات

- بعد تسجيل الدخول يُحفظ token ويُرسل مع الطلبات.
- `RequireAuth` يمنع فتح الصفحات المحمية دون جلسة صحيحة.
- اختيار `Workspace/Role` يحدد القسم الذي يستطيع المستخدم دخوله.
- الحماية في الواجهة لتحسين تجربة المستخدم، لكن الحماية الحقيقية يجب أن تبقى أيضًا في Laravel عبر `middleware` و `permissions`.

القسم الأول: لوحة الإدارة Admin

5. لوحة الإدارة والمسارات

المسار الأساسي `/admin` ويحتوي على:

- `.Dashboard`
- `.Departments`
- `.Employees`
- `.Warehouses`
- `.Items`
- `.Roles & Permissions`

تستخدم الصفحات Layout موحدًا يحتوي Sidebar و Topbar ومنطقة محتوى، مع مكونات مشتركة مثل بطاقات الإحصائيات، شريط البحث والتصفية، الجداول والنوافذ المنبثقة.

6. إدارة الأقسام Departments

الميزات

- عرض الأقسام.
- إنشاء قسم وتعديله وحذفه.
- عرض موظفي القسم.
- ربط موظف بقسم.
- تعديل علاقة الموظف بالقسم أو حذفها.
- تحديد موقع الموظف داخل القسم مثل Manager أو Supervisor أو Staff بحسب البيانات المدعومة.
- البحث، الإحصائيات، وحالات التحميل والخطأ.

كيف حققناها؟

تعرض الصفحة البيانات الموجودة في Redux. عند تنفيذ عملية، تستدعي `Async Thunk` وظيفة الـ API المناسبة، ثم يقوم الـ Slice بتحديث القائمة. علاقات الموظف بالقسم تعامل كـ Resource منفصل لأن الموظف قد يمتلك بيانات حساب مستقلة عن تكليفه ضمن قسم.

سؤال محتمل

لماذا لم نخزن اسم القسم نصًا داخل الموظف؟

لأن العلاقة يجب أن تعتمد على `department_id`. الاسم قابل للتعديل، أما المعرّف فهو المرجع الثابت ويمنع تكرار البيانات وعدم اتساقها.

7. إدارة الموظفين Employees

الميزات

- جلب قائمة الموظفين من الـ API.
- إنشاء وتعديل وحذف موظف.
- عرض الاسم والبريد والهاتف والأدوار والبيانات المفيدة التي يعيدها السيرفر.
- البحث والتصفية حسب الدور.
- نماذج Validation ورسائل نجاح وخطأ.

حقول الإنشاء/التعديل

تشمل بحسب العملية: الاسم الأول والأخير، البريد، الهاتف، العنوان، الجنس، كلمة المرور والحقول التي يسمح بها endpoint الموظف.

ملاحظة مهمة للمناقشة

لا يجوز اختراع Department من Role. مثلاً `Customer Service Staff` هو Role وليس بالضرورة كائن Department. إذا response الموظفين لا يحتوي علاقة القسم، نخفي عمود القسم أو نطلب من الـ backend إرجاع:

```
{
  "department": { "id": 2, "name": "Customer Service" },
  "position": "staff"
}
```

هذه نقطة تدل على احترام **API contract** وعدم تخمين علاقات غير موجودة.

8. المستودعات Warehouses

الميزات

- عرض المستودعات.
- جلب المواقع لاستخدامها في الاختيار.
- إنشاء وتعديل وحذف مستودع.
- ربط المستودع بـ `location_id` مع الاسم والعنوان والوصف.
- فتح تفاصيل العناصر المرتبطة بالمستودع.

طريقة الإرسال

يُبنى `FormData` يحتوي:

- `name`
- `location_id`
- `address` عند وجوده
- `description` عند وجوده

نفصل الموقع كمرجع بدل تكراره، وهذا تطبيق للعلاقات الطبيعية في قواعد البيانات.

9. العناصر والمخزون Items

الميزات

- CRUD كامل للعناصر.
- ربط العنصر بمستودع.
- البحث والعرض ضمن جدول موحد مع باقي لوحة الإدارة.
- إدارة كمية العنصر وحالته وتواريخ الشراء والاستلام والانتهاء.

الpayload الفعلي

- `warehouse_id`
- `sku`
- `name`

- description
- quantity
- status
- expiry_date
- purchase_date
- received_date

أسئلة محتملة

لماذا SKU مهم؟

هو رمز أعمال واضح للعنصر، بينما `id` هو معرف قاعدة بيانات داخلي.

كيف نتعامل مع مادة ليس لها تاريخ انتهاء؟

يُرسَل الحقل فارغًا/اختياريًا ويعرض - بدل تاريخ وهمي.

كيف نمنع `quantity` سالبة؟

Validation في الواجهة لتحسين التجربة، و Validation إلزامي في الـ backend لأنه مصدر الثقة.

10. الأدوار والصلاحيات Roles & Permissions

الميزات

- عرض الأدوار.
- إنشاء وتعديل وحذف Role.
- جلب جميع Permissions.
- تحديد Role ثم تعديل مجموعة صلاحياته.
- إسناد دور أو أكثر إلى مستخدم.
- واجهة مقسمة إلى Tabs: Roles, Permissions, Assign Roles

Endpoints المستخدمة

```
GET    /role
POST   /role
PUT     /role/{id}
DELETE /role/{id}
GET     /permission
POST    /role/selectPermission/{roleId}
POST    /role/assignRoles/{userId}
```

الصلاحيات تُرسل كمصفوفة `permissions[index]`، بينما إسناد الأدوار يرسل JSON بالشكل `{ roles: [1, 2]}`.

- Role مجموعة صلاحيات تعبر عن وظيفة، مثل Admin أو Customer Service Staff.
- Permission فعل واحد دقيق، مثل `read.warehouse` أو `update.warehouse`.
- المستخدم يُعطى Role، والRole يحتوي Permissions. هذا هو RBAC: Role-Based Access Control.

خطأ `guard_name`

إذا أعاد Laravel رسالة `Undefined array key guard_name` فهذا يعني أن عقد endpoint أو Validation في الbackend يتوقع الحقل رغم أن نموذج Postman قد يعرض حقولاً مختلفة. الحل الصحيح هو توحيد العقد: إما يجعل السيرفر `guard_name` اختياريًا بقيمة افتراضية مثل `web`، أو يصريح أنه حقل مطلوب ونرسله. لا ينبغي إخفاء خطأ backend على أنه نجاح في الواجهة.

القسم الثاني: خدمة العملاء Customer Service

11. المسارات والميزات العامة

المسار الأساسي `/customer-service` ويحتوي:

- Dashboard.
- Clients.
- Appointments.
- Orders، وتفاصيل طلب مستقل `/orders/:orderId`.
- Complaints.
- Lottery.
- FAQ Tree و Chat.

12. العملاء Clients

الميزات

- عرض العملاء القادمين من API.
- البحث ضمن المعلومات المتاحة.
- عرض بيانات الحساب والبيانات الإضافية بطريقة منظمة.
- تنفيذ العمليات التي يسمح بها السيرفر مثل تعطيل العميل.
- استخدام بيانات العميل في إنشاء موعد واختيار صاحب الطلب.

نفصل `account` عن `additional_info` لأن بيانات تسجيل الدخول مشتركة بين أنواع الحسابات، بينما تاريخ الميلاد والحالة الاجتماعية والرقم الوطني تخص ملف العميل.

13. المواعيد Appointments

العمليات المربوطة

```
GET /appointment
POST /appointment
PUT /appointment/{id}
PUT /appointment/cancel/{id}
PUT /appointment/markAsDone/{id}
```

إنشاء موعد

النموذج يرسل الحقول المناسبة لعقد الAPI:

- `client_id`
- `order_id`
- `av_slot_id`
- `type`
- `notes` عند وجودها

الواجهة تجلب العملاء والطلبات والـ `available slots`. تُعرض الأوقات المحجوزة بوضوح، ويجب منع اختيار `slot` محجوز قبل الإرسال، مع بقاء التحقق النهائي في السيرفر لتجنب حجزين مترامين.

تعديل الموعد

يستخدم `PUT /appointment/{id}` ونفس `builder` المرن للحقول غير الفارغة. تظهر الحقول التي يمكن تعديلها فعليًا بدل إرسال قيم وهمية.

الإلغاء والإكمال

- `Cancel` يغيّر حالة الموعد عبر `endpoint` مخصص.
- `Complete` يستخدم `markAsDone`.
- رفض إكمال موعد مستقبلي هو `Business Rule` صحيح من `backend`؛ تعرض الواجهة الرسالة ولا تحذف الجدول أو تحول الخطأ إلى `Empty State`.

لماذا لم يظهر اسم العميل سابقًا؟

قائمة المواعيد كانت تعيد `order` و `slot` والحالة والنوع، لكنها لا تعيد كائن `client`. لا يمكن استخراج اسم موثوق من `order.id` وحده. الحل الأفضل أن يعيد `backend` داخل كل موعد:

```
{
  "client": {
    "id": 1,
    "account": {
      "full_name": "Youssef El-Mansouri",
      "email": "...",
      "phone": "..."
    }
  }
}
```

يمكن استخدام cache مؤقت من بيانات الإنشاء، لكنه ليس بديلًا عن response كامل لأن البيانات تختفي على جهاز جديد أو بعد مسح التخزين.

14. الطلبات Orders

العمليات

- عرض جميع الطلبات.
- قراءة تفاصيل طلب واحد.
- جلب طلبات قسم.
- جلب طلبات الوحدات أو الحلول لعميل.
- تحويل الطلب إلى قسم.
- تغيير الحالة.
- إضافة ملاحظة وتعديلها وحذفها.

Endpoints الأساسية

```
GET /order
GET /order/{id}
GET /order/departmentOrders/{departmentId}
GET /order/getClientUnitOrders/{clientId}
GET /order/getClientSolutionOrders/{clientId}
PUT /order/transfer/{id}
PUT /order/changeStatus/{id}
POST /order/note/add/{id}
PUT /note/{noteId}
DELETE /note/{noteId}
```

العرض الديناميكي

الطلب قد يكون `unit` أو `solution` :

- إذا كان Unit نعرض رقم الوحدة والمساحة والطابق والغرف والسعر والحالة.
- إذا كان Solution نعرض اسم الخدمة ووصفها والسعر الأصلي والحالي والعرض والمرفقات.
- لا نعرض بطاقة Unit فارغة لطلب Solution؛ العرض يعتمد على `type` ووجود الكائن.
- معلومات العميل تؤخذ من `client.account` و `client.additional_info`.

قائمة Actions

زر النقاط يفتح Dropdown داخل صف الطلب لعمليات Change Status و Transfer Order و Add Note. يجب إدارة `z-index` و `overflow` بحذر؛ إذا كان الأب `overflow: hidden` فسُتُقص القائمة. الحل إما `overflow: visible` في حاوية الجدول المناسبة أو Portal للقوائم التي يجب أن تتجاوز حدود الجدول.

15. الشكاوى Complaints

الميزات

- عرض الشكاوى وحالاتها.
 - جلب أنواع الشكاوى.
 - تغيير حالة الشكاوى.
 - حذف شكاوى حسب صلاحية المستخدم.
 - إنشاء/حذف أنواع الشكاوى عند توفر الصلاحية.
 - البحث والتصفية وإظهار بيانات العميل والمحتوى المفيد.
- استخدام endpoint منفصل للأنواع يسمح بإضافة نوع جديد دون تعديل كود الواجهة في كل مرة.

16. القرعة Lottery

العمليات المربوطة

```
GET /lottery
GET /lottery/{id}
POST /lottery
PUT /lottery/{id}
PUT /lottery/cancel/{id}
PUT /lottery/drawWinner/{id}
```

إنشاء وتعديل القرعة

يُرسَل:

- عنوان القرعة.
 - `unit_id` كرقم.
 - مصفوفة `rules` ، وكل Rule تحتوي `rule_key` , `operator` , `rule_value` .
- الـ `rule_key` يُختار من قائمة لمنع أخطاء الكتابة. عند `social_status` تظهر قائمة بالقيم المدعومة: `single` , `married` , `divorced` . أما العمر أو الدخل/الراتب فيدخل المستخدم قيمة رقمية، والعمر يُكتب بالسنوات.

رسم الفائز

قبل الرسم تتحقق الواجهة من:

- أن الحالة تسمح بالرسم، عادة `open` .
- وجود مشاركين مؤهلين.
- عدم تنفيذ الرسم مرتين على قرعة مكتملة.

القرار النهائي والفائز يأتيان من السيرفر، وليس من `Math.random()` في المتصفح، حتى تكون العملية موثوقة وغير قابلة للتلاعب. بعد نجاح `endpoint`:

- تُحدَّث حالة Redux والقائمة.
- تُجلب التفاصيل عند الحاجة للحصول على اسم الفائز الكامل.
- تظهر مرحلة تشويق ثم Reveal للفائز.
- تستخدم الواجهة CSS animations وconfetti وعناصر زخرفية.
- يُولد Web Audio API مؤثرات الرسم والفوز دون الاعتماد على ملف صوت خارجي.
- يوجد منع لإعادة بدء animation عند وصول نفس تحديث القرعة مرة ثانية.

لماذا قد تظهر Completed ثم رسالة فشل؟

قد ينجح أول request ويصبح `completed` status، ثم ينطلق request ثانٍ بسبب نقر متكرر أو state/effect مكرر؛ السيرفر يرفض الثاني لأنه لا يرسم قرعة مكتملة. نعالج ذلك بتعطيل الزر أثناء `drawing` ، منع الطلب المكرر، وعدم إظهار زر Draw للحالات غير المفتوحة.

17. الدردشة Chat

REST API

```
GET /chat/rooms
GET /chat/unassigned
POST /chat/rooms/{roomId}/claim
GET /chat/rooms/{roomId}/messages
POST /chat/message
```

كيف تعمل الدردشة؟

1. نجلب الغرف المسندة وغير المسندة.

2. يختار الموظف غرفة أو يعمل Claim لغرفة غير مسندة.
3. نجلب سجل الرسائل عبر REST.
4. الإرسال يتم عبر REST لضمان حفظ الرسالة في قاعدة البيانات.
5. الاستقبال اللحظي يتم عبر Laravel Echo و Pusher على private channel خاص بالغرفة.
6. عند وصول event نتحقق من الغرفة ونضيف الرسالة إلى Redux مع منع التكرار.
7. نتحدث آخر رسالة وترتيب المحادثة ويعمل auto-scroll إلى آخر رسالة.

لماذا REST مع WebSocket معًا؟

- REST مناسب لجلب التاريخ وحفظ الرسالة مع response واضح.
- WebSocket مناسب لدفع الرسالة الجديدة فورًا دون polling.
- الجمع بينهما يعطي موثوقية التخزين وتجربة لحظية.

الصور والصوت والتمرير

- تعرض الصورة من حقول avatar المحتملة إذا أعادها API، مع fallback إلى Initials عند غيابها أو فشل تحميلها.
- الصور داخل الرسائل تحتاج endpoint رفع/مرفقات؛ لا يمكن للواجهة اختراع رابط ملف لا يخزنه السيرفر.
- حاوية الرسائل لها scroll مستقل، وعند رسالة جديدة نستخدم ref و `scrollIntoView`.
- Web Audio API يولّد صوتًا مختلفًا للرسالة الواردة والصادرة، مثل تطبيقات المحادثة.
- الصوت قد لا يعمل قبل أول تفاعل للمستخدم بسبب سياسة Autoplay في المتصفحات، وهذا قيد من المتصفح وليس خطأ في React.

Realtime Security

- Echo يستخدم Bearer token في broadcasting auth.
- private channel يجب أن يتحقق في Laravel أن المستخدم مخول لدخول الغرفة.
- إخفاء غرفة من UI ليس حماية؛ authorization في السيرفر إلزامي.

18. شجرة الأسئلة الشائعة FAQ Decision Tree

العمليات

```
GET    /faqs/admin/tree
POST   /faqs
PUT    /faqs/{id}
DELETE /faqs/{id}
```

أنواع العقد

- `category` : تصنيف يمكن أن يحتوي أبناء.
- `answer` : إجابة نهائية؛ يستخدم `content` لعرض التفاصيل.

- `action_human` : نهاية تنقل العميل إلى موظف بشري.

بناء الشجرة

- الجذر يرسل `parent_id = null`.
- الابن يرسل ID العقدة الأب.
- Component recursive يعرض أي عمق دون كتابة UI منفصل لكل مستوى.
- التصنيفات تظهر Accordion مع عدد الأبناء.
- الضغط على سؤال يفتح الإجابة بوضوح.
- يوجد Search ضمن العناوين والمحتوى.
- الإضافة والتعديل والحذف متاحة من مكان العقدة المناسب.

لماذا Recursive Component؟

لأن عمق الشجرة غير ثابت. الدالة تعرض Node ثم تستدعي نفس Component لكل child. بذلك يدعم التصميم مستوى واحدًا أو عشرة مستويات دون تكرار الكود.

19. الـ State Management باستخدام Redux Toolkit

كل Slice تحتوي عادة:

- البيانات مثل `rooms`, `orders`, `items`.
- `loading` أثناء الطلب.
- `error` عند الفشل.
- بيانات محددة مثل `selectedOrder` أو `selectedRoom`.
- reducers محلية للتحديد أو إضافة realtime message.
- `extraReducers` لمعالجة pending/fulfilled/rejected الخاصة بالـ thanks.

لماذا Redux وليس useState فقط؟

نستخدم Redux عندما تكون البيانات مشتركة، لها عدة عمليات async، أو يجب تحديثها من WebSocket ومن أكثر من Component. نستخدم `useState` للحالة المحلية القصيرة، مثل فتح Modal أو نص search أو العقدة المفتوحة في FAQ.

منع التكرار في الرسائل

الرسالة المرسله قد تصل مرة من response ومرة من WebSocket. لذلك نقارن ID، أو مفتاحًا بديلًا محسوبًا من الغرفة والمرسل والنص والوقت، قبل الإضافة.

20. التصميم وتجربة المستخدم

المكونات المشتركة

- Layout و Sidebar و Topbar.
- Toolbar للبحث والتصفية وأزرار الإضافة.
- StatCard للإحصائيات.
- TableCard للجداول.
- Modal عبر Portal لتجنب مشاكل القص و z-index.
- Status badges و dropdowns.
- Empty, Loading, Error states.

Dark / Light Mode

تتم إدارة الثيم مركزيًا، وتُضبط قيمة `data-theme` أو class على عنصر عام. تعتمد CSS على variables بدل كتابة لون ثابت في كل Component، وتحفظ القيمة في `localStorage`.

العربية والإنجليزية

- قاموس ترجمة مركزي و Hook للحصول على النص.
- تخزين اللغة المختارة.
- تغيير `dir` إلى `rtl` للعربية و `ltr` للإنجليزية.
- إرسال `Accept-Language` إلى السيرفر.
- استخدام CSS منطقي قدر الإمكان مثل `inline-start/end` لتقليل شروط الاتجاه.

Accessibility

- استخدام buttons حقيقية بدل div قابل للنقر.
- labels للنماذج و `aria-label` للأزرار الأيقونية.
- إغلاق Modal بزر Escape ومنع scroll خلفه.
- حالات disabled أثناء الطلب.
- عدم الاعتماد على اللون وحده لشرح الحالة.

21. الإشعارات

يوجد دعم لـ Firebase وواجهات FCM:

- تسجيل device token عبر `/device-tokens`.
- حذف token عند Logout.
- جلب الإشعارات.
- جلب unread count لجرس الإشعارات.
- تحديد إشعار واحد أو الجميع كمقروء.

الصوت يمكن إنشاؤه عبر Web Audio API. ويجب طلب إذن Notification من المستخدم وعدم افتراض الموافقة.

22. معالجة الأخطاء والحالات الطرفية

- فصل `loading` عن `error` عن القائمة الفارغة.
- فشل Mutation لا يمسح القائمة الناجحة الموجودة.
- رسائل Laravel تُحوّل إلى نص مفهوم.
- منع `double submit` بتعطيل الزر أثناء الطلب.
- نتحقق من `null` قبل قراءة `nested fields`.
- الواجهة لا تخمّن حقولاً لا يعيدها API.
- Validation في Frontend لتجربة المستخدم، وفي Backend للأمان وسلامة البيانات.

23. أسئلة تقنية متوقعة مع أجوبة مختصرة

1. ما الفرق بين Page وComponent؟

Component جزء قابل لإعادة الاستخدام، مثل Modal أو Page. StatCard تنسّق عدة Components وتمثل route كاملة.

2. ما الفرق بين State و Props؟

Props تأتي من الأب ولا يعدلها الابن مباشرة. State بيانات داخلية متغيرة تسبب إعادة الرسم عند تحديثها.

3. ما وظيفة `useEffect`؟

تشغيل side effects مثل جلب البيانات أو الاشتراك بقناة، مع `cleanup` لإلغاء الاشتراك. يجب ضبط dependencies لمنع loop أو طلبات مكررة.

4. لماذا نستخدم `useMemo` و `useCallback`؟

لتثبيت قيمة محسوبة أو function عندما يكون ذلك مفيداً لتقليل حسابات أو renders غير لازمة، وليس لاستخدامهما في كل مكان عشوائياً.

5. ما هو Async Thunk؟

Action غير متزامن في Redux Toolkit يمر بحالات `pending` و `fulfilled` و `rejected`، ويتيح للـ `Slice` تحديث `loading/data/error`.

6. ما الفرق بين PUT و PATCH؟

PUT يُستخدم عادة لاستبدال/تحديث resource وفق عقد كامل، و PATCH لتعديل جزئي. الاستخدام النهائي يتبع تصميم API المتفق عليه.

7. لماذا نستخدم IDs بدل الأسماء في العلاقات؟

لأن ID فريد وثابت وأصغر، بينما الاسم قد يتكرر أو يتغير.

8. كيف حميتم الtoken؟

نرسله Bearer عبر HTTPS ولا نطبعه صريحًا. التخزين الحالي في localStorage عملي لكنه معرض لـXSS؛ في إنتاج عالي الحساسية يمكن استخدام HttpOnly Secure SameSite cookie مع CSRF protection.

9. كيف منعتم المستخدم غير المخول؟

Route guards في الواجهة، ويفترض permissions + Laravel middleware في السيرفر. الواجهة وحدها ليست حازر أمان.

10. ما هو CORS؟

سياسة متصفح تتحكم بوصول Frontend على origin إلى Backend على origin آخر، ويضبطها السيرفر للمصادر والـheaders والطرق المسموحة.

11. ما الفرق بين Authentication وAuthorization؟

Authentication يثبت من المستخدم، وAuthorization يحدد ماذا يستطيع أن يفعل.

12. كيف تعمل المحادثة الفورية؟

نحفظ ونقرأ عبر REST، ونستقبل الأحداث الجديدة عبر private WebSocket channel باستخدام Pusher وLaravel Echo، ثم نحدّث Redux.

13. لماذا لا نختار فائز القرعة في Frontend؟

لأن العميل قابل للتلاعب. السيرفر يطبق الشروط ويختار ويسجل النتيجة ضمن عملية موثوقة.

14. كيف تمنعون Race Condition في الحجز؟

الواجهة تمنع اختيار slot ظاهر كمحجوز، لكن السيرفر يجب أن يستخدم transaction/unique constraint أو lock لأن مستخدميهم قد يحجزان في اللحظة نفسها.

15. ما فائدة API normalization؟

بدل أن تتعامل كل صفحة مع شكل Axios مختلف، تحصل جميعها على `ok/status/message/data`، فيصبح الكود أبسط ومتناسقًا.

16. كيف تدعمون Responsive Design؟

Grid/Flexbox، قياسات مرنة، scroll، media queries أفقي للجدول عند الحاجة، وتغيير توزيع الأعمدة على الشاشات الصغيرة.

17. كيف تعمل شجرة FAQ؟

كل عقدة تحمل `parent_id` ونوعًا، والسيرفر يعيد React children يعرضها recursively، والتصنيفات تُفتح كـaccordion والإجابات تظهر عند الاختيار.

18. لماذا قد لا يعمل الصوت مباشرة؟

المتصفح يمنع Web Audio قبل تفاعل المستخدم. بعد أول click يمكن resume للـAudioContext وتشغيل المؤثرات.

19. كيف تختبرون المشروع؟

نختبر build وESLint، ونختبر سيناريوهات CRUD والـ validation والـ API errors والأدوار والاتجاهين والـ themes والـ realtime. الأفضل مستقبلاً إضافة Vitest/React Testing Library واختبارات End-to-End مثل Playwright.

20. ما أهم تحسينات Production؟

- إلغاء أو تقييد console logs.
- Error Boundary وcentral monitoring.
- اختبارات آلية.
- Pagination وserver-side search للقوائم الكبيرة.
- Lazy loading للصفحات.
- توثيق OpenAPI لعقد الـ API.
- حماية token أقوى وسياسة CSP.
- Retry مدروس للطلبات القابلة للإعادة فقط.

24. سيناريو Demo مقترح أمام اللجنة

1. تسجيل الدخول واختيار Workspace مناسب.
2. تبديل اللغة والธีม لإظهار التخصيص المركزي.
3. في Admin: إنشاء قسم أو موظف، ثم شرح Redux → API → Slice.
4. فتح Roles & Permissions وشرح RBAC.
5. في Customer Service: فتح طلب Solution وبيان العرض الديناميكي بدل Unit فارغة.
6. إنشاء موعد من عميل وطلب ووقت متاح، ثم شرح منع الحجز المزدوج.
7. إنشاء قرعة بشروط، عرض المشاركين، ثم رسم الفائز من السيرفر مع animation.
8. فتح Chat، إرسال رسالة، وشرح REST + WebSocket.
9. فتح FAQ Tree وإضافة child مع parent_id ثم عرض recursion.
10. إنهاء العرض بذكر فصل المسؤوليات، معالجة الأخطاء، وما يمكن تحسينه مستقبلاً.

25. قاعدة ذهبية أثناء المناقشة

لا تقولوا إن الواجهة تنفذ قاعدة أمان أو قرار أعمال حساس وحدها. قولوا دائماً:

الواجهة تتحقق لتحسين تجربة المستخدم، لكن السيرفر يعيد التحقق لأنه مصدر الحقيقة، والواجهة تعرض النتيجة القادمة من الـ API.

هذا ينطبق على الصلاحيات، اختيار الفائز، حجز الموعد، تغيير حالات الطلبات، ورفع الملفات.